Open Access

Scaling ITK's TBB Threader and v4 Image Metrics

Diagnosis and fixes from BRAINSTools and ANTs workloads

Hans J. Johnson[1]

[1]Department of Electrical and Computer Engineering, University of Iowa

Abstract

ITK selects the oneTBB threader by default when it is built with TBB. On a 48-thread workstation, eight BRAINSTools and ANTs regression tests ran 1.2 to 7 times slower under that default than under ITK's Pool threader. We traced the gap to three defects. The TBB region splitter divides work down to single voxels. The v4 image metric threaders inherit 16 work units per thread and keep per-work-unit state that is allocated, zeroed, and reduced on every evaluation. The Mattes mutual information derivative buffers have no memory bound, so a single sampled evaluation with a BSpline sized like the BRAINSFit default peaks at 30 GB. Three small ITK changes address these defects. With them, rigid and affine registration under TBB runs twice as fast as the unchanged Pool threader, and Mattes peak memory for sampled BSplines falls from up to 30 GB to under 2.2 GB. For a dense 3,993-parameter BSpline it falls from 24 GB to 3.1 GB, at a cost of about 8% in time. On 512^3 images the gains grow: a sampled BSpline evaluation that peaked at 183 GiB under upstream TBB uses 6.1 GiB and runs 26% faster. The test suites of ANTs, BRAINSTools, and elastix show no regressions under either threader. Downstream timings were repeated ten times on an idle host, and the repeat count of every benchmark is stated with its result. This paper comes with the ITK patches, the benchmark sources, the harness scripts, and the raw measurements needed to reproduce each number.

Keywords ITK, multithreading, oneTBB, image registration, mutual information, performance

1. *Introduction*

ITK parallelizes filters through `MultiThreaderBase`, which has three back ends: Platform (one thread per work unit), Pool (a persistent thread pool), and TBB (oneTBB tasks). When ITK is built with `Module_ITKTBB`, TBB is the default. BRAINSTools and ANTs inherit that default, and both projects saw their registration tests slow down by a large factor when they moved to it.

This paper does three things. It describes how the slowdown was measured. It identifies three failure modes in ITK, each with the instrument that found it. It reports the effect of three changes, proposed on branch `perf-tbb-workunits-grain`, on both micro and downstream benchmarks.

Published Nov 03, 2026

2. *Experimental setup*

2.a. *Host and builds:*

All measurements ran on one workstation: an Intel Xeon w7-3545 (24 cores, 48 hardware threads, one socket), Ubuntu 24.04, GCC 13.3, oneTBB 2022.3.0. ITK was built statically in Release mode inside the BRAINSTools SuperBuild, which also builds ANTs and its standalone test tree against the same ITK. Two ITK baselines were used, so that results hold for both maintained branches:

- ITK main at `003e7538e15`;
- ITK release-5.4 at `eef984f4982`.

For each baseline, the ITK source in the SuperBuild was switched to each configuration under test, and ITK, ANTs, BRAINSTools, and the ANTs tests were rebuilt incrementally (`scripts/rebuild_superbuild.sh`).

2.b. *Downstream tests:*

Eight tests were chosen from a full BRAINSTools and ANTs run as the ones with the largest TBB slowdown. They cover the constellation detector (BCD), mid-sagittal plane alignment, BRAINSFit rigid and BSpline registration, antsRegistration rigid and affine registration, and two ANTS symmetric normalization tests. The threader was selected with `ITK_GLOBAL_DEFAULT_THREADER`.

2.c. *Protocol:*

`scripts/repeat_selected.py` runs each test alone with `ctest -j1`. Before every run it waits until the host is at least 90% idle over 3 seconds, read from `/proc/stat`. Tests cycle in rounds, so drift spreads across all of them, and each configuration runs ten rounds. Every run's start time, status, and idle fraction is recorded. Runs that overlapped other CPU work on the host, listed in `data/excluded_windows.csv`, were excluded: seven of 400 on release-5.4 and none of 400 on main. All 800 runs passed.

2.d. *Microbenchmarks:*

Two C++ programs in `src/` isolate the threader and the metric. `itkTBBBench.cxx` times `ParallelizeImageRegion`, `ParallelizeArray`, and a 3D Euler resampling at 16^3 to 128^3 voxels, and counts the chunks each call produces. `mattesBench.cxx` times one Mattes MI v4 `GetValueAndDerivative` on 96^3 images with 50 histogram bins, for an affine transform or a BSpline inside a `CompositeTransform`. It reports the work units used, wall and CPU time, and peak RSS.

3. Diagnosis

3.a. *Failure mode 1: TBB splits regions and arrays too finely:*

`TBBImageRegionSplitter::is_divisible()` returns true until every dimension of a chunk has size one. Under the auto partitioner with 48 workers stealing, a 16^3 region was split into 2,124 chunks, the smallest a single voxel, where Pool used 16 slices. `ParallelizeArray` ran one TBB task per element, and each task built its own progress reporter. In the first benchmark run (`data/bench_main_48thr_20261102.txt`), TBB spent 251 microseconds per `ParallelizeImageRegion` call on a 16^3 region against 83 for Pool, and used 143 s of

CPU for 3.1 s of wall time, with 47 cores busy scheduling tiny tasks. Figure 1 repeats the benchmark three times on both trees.

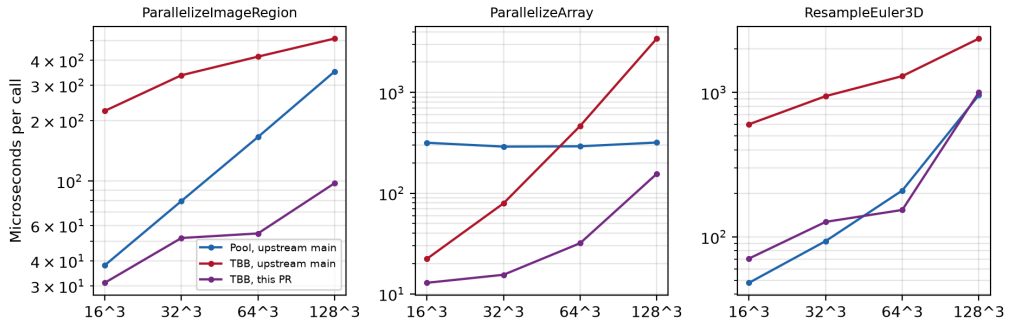


Figure 1: Time per call for three threader kernels, 48 threads, median of three runs. Source: `data/summary_microbench.csv`.

3.b. *Failure mode 2: image metric threaders inherit 16 work units per thread:*

`TBBMultiThreader` sets 16 work units per thread, 768 on this host, and `DomainThreader` copies that count. Every v4 image metric threader keeps one derivative per work unit, and Mattes MI also keeps one joint PDF and one derivative buffer per work unit. All are allocated, zeroed, and reduced on every evaluation. With 5% sampling, Mattes used 763 work units under TBB and 128 under Pool. Setting work units to the thread count through `SetMaximumNumberOfWorkUnits` was silently ignored; that defect was fixed separately in ITK pull request 7080.

3.c. *Failure mode 3: Mattes derivative buffers have no memory bound:*

For transforms without local support, which in Mattes MI v4 includes a BSpline, each work unit buffers one derivative row per sample, one value per parameter, and reduces the buffer under a mutex. The buffer starts at 500 rows and doubles under lock contention up to 8,000 rows. At 3,993 BSpline parameters this reached 25 GB with dense sampling. At 10,125 parameters, close to the BRAINSFit default grid, one sampled evaluation peaked at 30,066 MiB of RSS on upstream main.

4. Changes

Change	Where	Effect
Sampled metrics: at most one work unit per thread	Metric threader base constructor, sampled-point partitioner only	Default only; dense metrics keep one work unit per slab; an explicit work-unit count still applies; other domain threaders unchanged

Buffer limited to 32 MiB per work unit	Mattes derivative buffer manager	Starts within the budget and doubles only while the doubled buffer fits; transforms with up to 524 parameters keep their previous sizes
Chunks of about pixels per work unit	`TBBMultiThreader`	Region chunks keep rows whole unless the region is one-dimensional; `ParallelizeArray` runs contiguous ranges

Table 1: The three changes on branch `perf-tbb-workunits-grain`.

The buffer budget was chosen from a sweep (Figure 2). A 4 MiB budget minimized memory but slowed sampled BSpline evaluations by about 20%. The variant labeled 16 MiB in that sweep could double once, so its buffers reached about 32 MiB per work unit; the final change enforces 32 MiB as a hard limit and reproduces those buffer sizes.

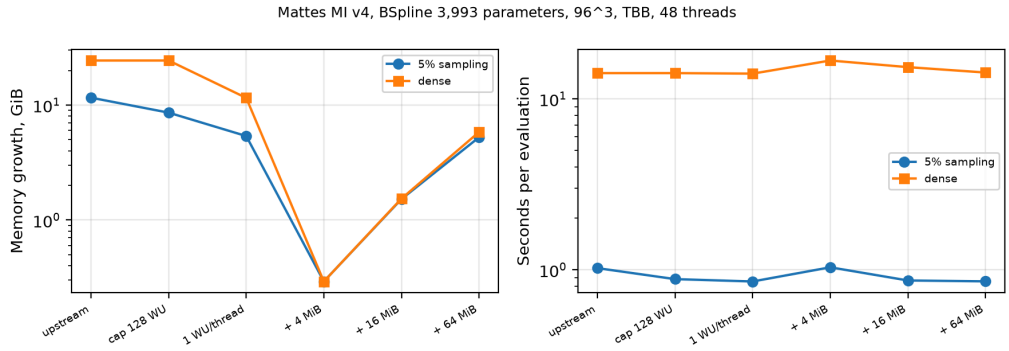


Figure 2: Mattes MI v4 memory growth and time per evaluation for a 3,993-parameter BSpline under TBB, for the upstream code and five candidate changes. Median of three runs. The metric value was identical in every run.

5. *Results*

5.a. *Downstream tests:*

Figure 3 and Table 2 give the medians of ten runs on ITK main. With all three changes, BRAINSFit rigid, antsRegistration rigid, and antsRegistration affine under TBB run 8 to 14 times faster than before, and about twice as fast as the unchanged Pool threader. The TBB chunking change alone halves BCD and cuts mid-sagittal plane alignment by a quarter. BRAINSFit BSpline is within run-to-run noise in every configuration. Release-5.4 agrees with main to within about 2 seconds on every test except BRAINSFit BSpline (Figure 4).

Test	Before, TBB	Before, Pool	Chunking only, TBB	All, TBB	All, Pool
BCD	60.7	18.4	27.7	27.3	18.0
AlignMSP	26.5	16.1	19.7	19.7	15.4
BRAINSFit rigid	13.8	3.7	13.9	1.7	1.8
BRAINSFit BSpline	73.7	61.9	71.2	62.3	63.2
ANTs rigid	25.3	3.9	25.7	1.8	1.7
ANTs affine	10.9	1.5	10.7	0.8	0.7
ANTS rot GSyN	25.2	13.5	22.4	20.4	13.1
ANTS rot Exp	17.7	8.2	19.6	14.0	7.6

Table 2: Median seconds over ten runs, ITK main. Source: `data/summary_downstream_main.csv`.

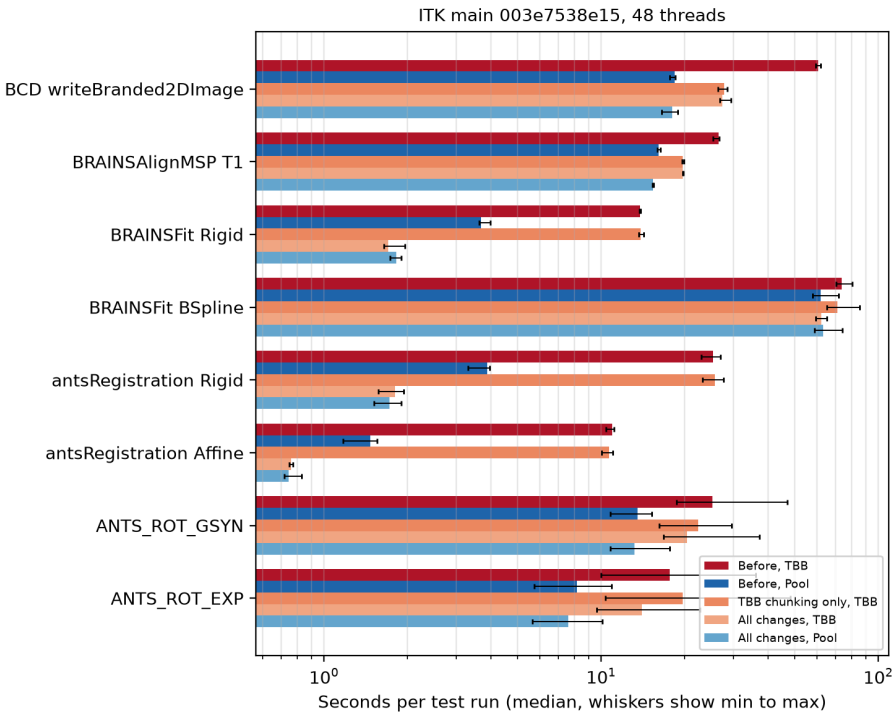


Figure 3: Downstream tests on ITK main, 48 threads, log scale. Bars are medians of ten runs; whiskers span the fastest to the slowest run.

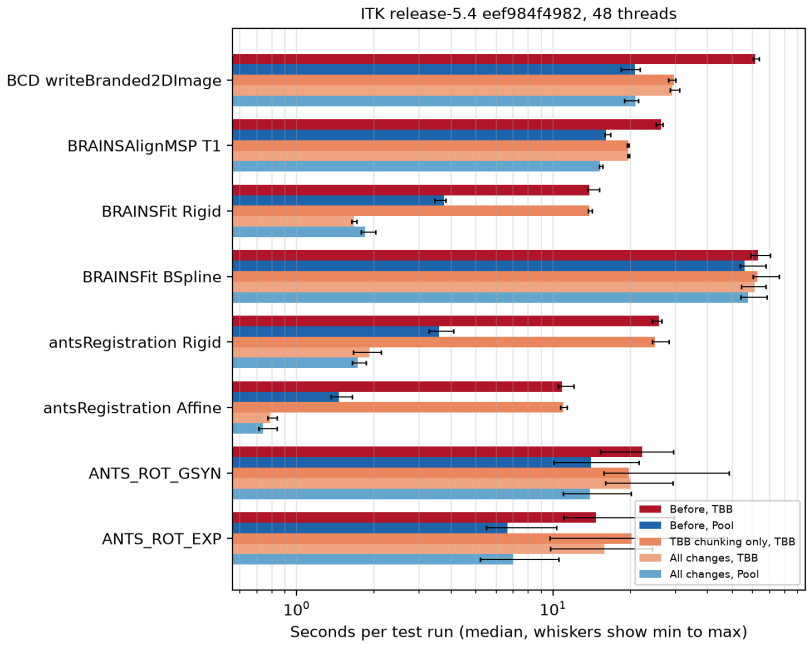


Figure 4: The same tests on ITK release-5.4.

The two ANTS symmetric normalization tests vary by up to 19 seconds between runs of the same configuration, so single runs of them cannot rank configurations. Their medians show no regression from the chunking change.

5.b. *Mattes MI v4 on the final branch:*

Table 3 compares upstream main with the final branch for Mattes MI v4 on 96^3 images at 48 threads, median of three runs. Peak RSS falls by 5 to 34 times for BSplines and by up to a third for affine transforms. Sampled evaluations get faster, most for affine transforms under TBB (7.0 to 2.5 ms). Dense BSpline evaluations get 7% to 11% slower, because each work unit's buffer now reduces under the mutex more often. The metric value is identical to all printed digits in every case. These measurements predate restricting the work-unit cap to sampled metrics; the sampled rows are unchanged by that restriction, and the dense rows now use upstream's one work unit per slab (96 here), as described under Figure 6. A `pre-commit` run at low priority overlapped part of the first repeat (`data/interference_windows.txt`); dropping that repeat moves no median by more than 1.3%.

Threader	Transform	Sampling	Work units	Time, ms	Peak RSS, MiB
TBB	Affine, 12	5%	763 / 48	7.0 / 2.5	85 / 65
TBB	Affine, 12	dense	96 / 48	28.7 / 27.2	151 / 99
TBB	BSpline, 3,993	5%	763 / 48	1,025 / 866	11,887 / 2,168
TBB	BSpline, 3,993	dense	96 / 48	14,227 / 15,471	25,117 / 2,118

TBB	BSpline, 10,125	5%	763 / 48	2,230 / 2,135	30,066 / 1,773
TBB	BSpline, 10,125	dense	96 / 48	35,438 / 39,440	60,322 / 1,773
Pool	Affine, 12	5%	128 / 48	3.5 / 2.2	68 / 63
Pool	BSpline, 3,993	5%	128 / 48	886 / 858	9,519 / 2,170
Pool	BSpline, 10,125	5%	128 / 48	2,136 / 2,117	20,346 / 1,777
Pool	BSpline, 10,125	dense	96 / 48	35,389 / 39,052	60,380 / 1,774

Table 3: Mattes MI v4 per `GetValueAndDerivative`, upstream main against this branch, 48 threads. Source: `data/summary_mattes_compare.csv`.

5.c. *Downstream tests on the final branch:*

The tables above were measured before review narrowed the work-unit change to the image metric threaders and made the buffer budget a hard limit. The final branch was measured again against its own base commit, upstream `f77bbc9038c`, with peak RSS recorded per test through GNU `time` (Table 4, Figure 5). An older baseline, `003e7538e15`, gave the same medians to within the run-to-run spread, so the 47 upstream commits between the two do not explain the changes. All 240 runs passed.

Test	Base TBB	Branch TBB	Base Pool	Branch Pool	Peak RSS, MiB, base / branch
BCD	60.0	27.4	18.3	18.2	300 / 303
AlignMSP	24.9	20.6	15.4	16.1	86 / 88
BRAINSFit rigid	13.8	1.8	3.7	1.8	180 / 70
BRAINSFit BSpline	74.8	62.5	63.0	64.3	3,431 / 744
ANTs rigid	24.6	1.9	3.8	1.8	304 / 80
ANTs affine	10.7	0.8	1.5	0.75	333 / 98
ANTS rot GSyN	21.1	18.2	11.6	14.8	273 / 273
ANTS rot Exp	16.3	17.4	6.3	7.1	201 / 201

Table 4: Final branch against its base commit, ITK main, 48 threads. Median seconds (5 runs for the base, 10 for the branch) and median peak RSS. Source: `data/summary_downstream_rss.csv`.

Peak RSS is for the TBB runs. Under Pool it falls from 85 to 69 MiB for BRAINSFit rigid, 101 to 81 MiB for antsRegistration rigid, 112 to 98 MiB for antsRegistration affine, and 3,819 to 753 MiB for BRAINSFit BSpline. The two ANTS normalization

tests span 5 to 18 seconds between runs of one configuration, so their medians do not separate the configurations.

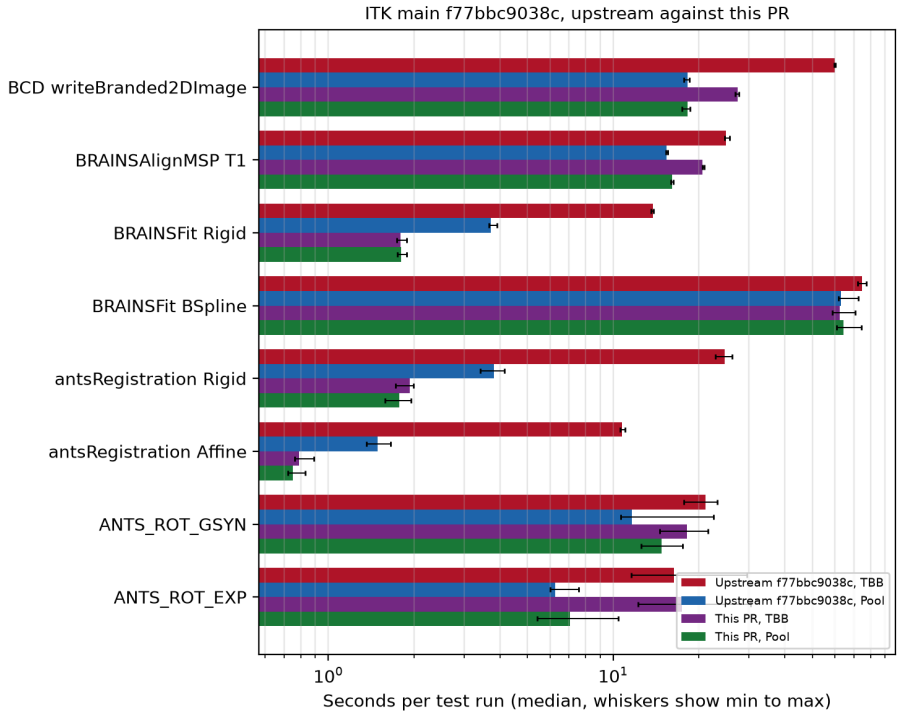


Figure 5: Final branch against its base commit, ITK main, 48 threads.

5.d. *Threader microbenchmarks on the final branch:*

On the final branch a 16^3 region becomes 256 chunks with 768 work units, one per image row, and 32 chunks with 48 work units; upstream made 2,078 and 3,070, with single-voxel chunks in both cases (`data/summary_chunks.csv`). Figure 1 now includes the branch. TBB `ParallelizeImageRegion` falls from 224 to 31 microseconds at 16^3 and from 513 to 98 at 128^3, faster than Pool at every size. `ParallelizeArray` falls from 3,427 to 156 microseconds at 128^3. Resampling at 16^3 and 32^3 remains slower than Pool (71 against 48, and 127 against 93 microseconds).

5.e. *Mattes MI v4 across thread counts:*

Figure 6 repeats the Mattes comparison at 1, 8, 24, and 48 threads (two runs each), again before the cap was restricted to sampled metrics. Peak RSS on the branch stays under 2.2 GB in every case; upstream reaches 60 GB at 8 threads for the 10,125-parameter BSpline. Single-threaded evaluations of the 3,993- and 10,125-parameter BSplines get 14% to 23% faster, consistent with the smaller buffers staying in cache; the 1,029-parameter BSpline is up to 7% slower. Dense BSpline evaluations at 8 or more threads get 2% to 11% slower.

One case regressed in the version measured above, which capped dense metrics too. A densely sampled affine metric at 8 threads under TBB went from 29 to 61 ms per

evaluation: the dense threader splits the image into slabs, and 8 slabs of unequal cost cannot be rebalanced, where upstream used 96. Raising the cap to 2 or 4 work units per thread recovered only part of it (49 and 39 ms) while multiplying BSpline memory (`data/mattes_wu_cap_sweep_20261103.tsv`). Every downstream gain came from sampled metrics (ANTs `Regular` 10% sampling, BRAINSFit 30% to 50%), so the final branch caps only the sampled-point threader. With that change the dense affine metric at 8 threads under TBB takes 31 ms against 30 ms upstream, and a dense 3,993-parameter BSpline peaks at 3.1 GB against 24 GB upstream, about 8% slower (`data/mattes_sparse_cap_check_20261103.tsv`, one repeat).

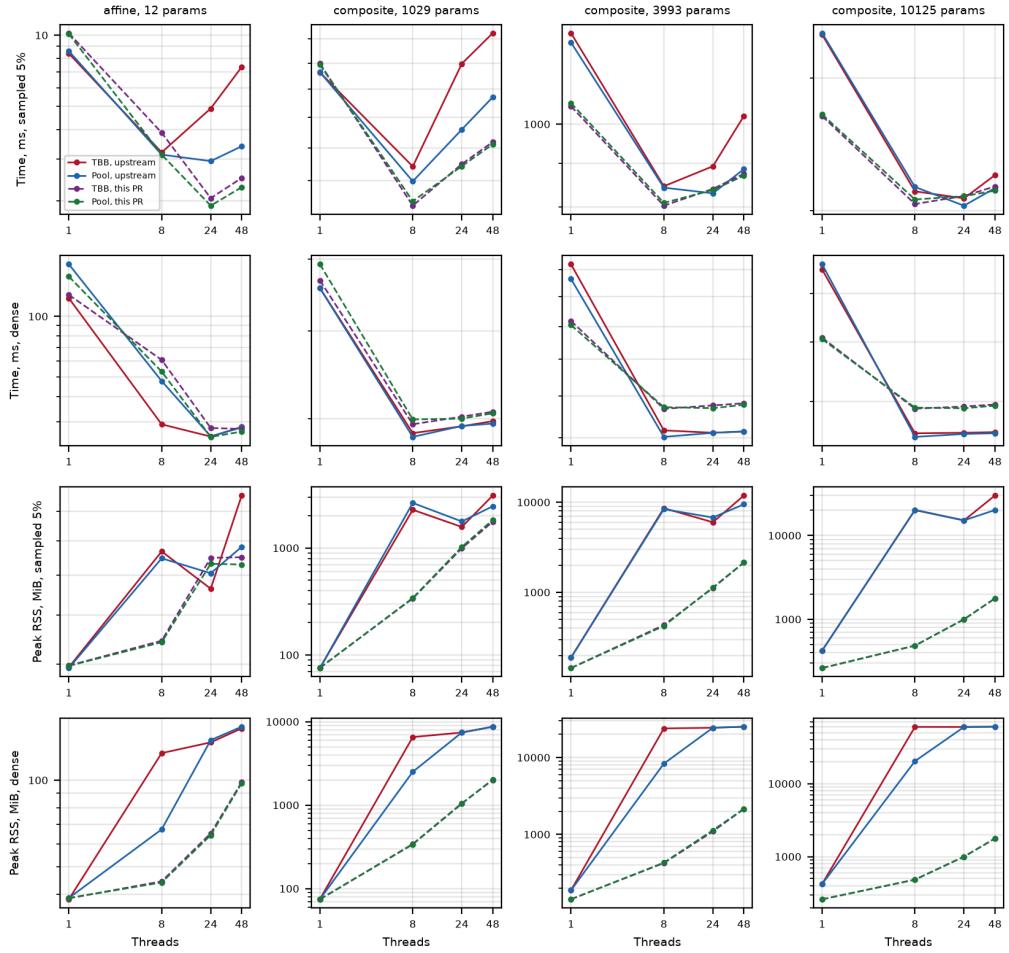


Figure 6: Mattes MI v4 time (top) and peak RSS (bottom) against thread count, upstream main and the final branch, both threaders. Columns are transform size and sampling. Median of two runs.

5.f. *Downstream test suites:*

The forest build testbed (`build_forest-itk-main`) built ITK with `Module_ITKTBB=ON` at the branch base `f77bbc9038c` and at the branch head `d0a1d85dbe5` (rebased afterward onto `fd19f432272` without changes to the patches; the final commits are `1d27b6a4701`, `973dc91aebd`, and `d8ee1791ebb`), rebuilt ANTs, BRAINSTools, and

elastix against each, and ran each project's full test suite with `ITK_GLOBAL_DEFAULT_THREADER` set to TBB and to Pool (per-test timeout 1,500 s). Table 5 shows the result: the same tests pass and the same tests fail before and after the change, under both threaders. The five failing tests fail identically at the base, so they predate this work.

Project	Threader	Base `f77bbc9038c`	Branch `d0a1d85dbe5`	Failing tests (both)
ANTs	TBB	185 / 185	185 / 185	none
ANTs	Pool	185 / 185	185 / 185	none
BRAINSTools	TBB	452 / 455	452 / 455	three Versor float tests
BRAINSTools	Pool	452 / 455	452 / 455	same three
elastix	TBB	162 / 164	162 / 164	two: Transformix test and example image compare
elastix	Pool	162 / 164	162 / 164	same two

Table 5: Test suites of downstream projects, ITK base against this branch. Source: `data/forest_ctest_summary.csv` and the logs in `data/forest_pr21/`.

Inside ITK, 797 tests in ITKCommon, ITKMetricsv4, ITKOptimizersv4, ITKRegistrationMethodsv4, and ITKLevelSetsv4 pass under both TBB and Pool on the final branch, after the rebase.

The three BRAINSTools failures are one pre-existing ITK defect: `itk::Versor<float>::Set(matrix)` checks orthogonality against a tolerance of 1e-7, below single-precision rounding, and rejects a valid rotation. `ConvertVersorRigidToVersorRigidInfloat` aborts, and the other two tests read its output. The test passes with Ubuntu GCC 13.3 and fails with the testbed's conda GCC 14.3, at the base and at the branch alike. It is fixed separately.

6. *Discussion*

The three failure modes add up. Fine chunking dominates kernels that run once per filter, such as resampling in BCD. Work-unit state dominates registration, where the metric runs hundreds of times per level and each evaluation is short. The buffer limit matters only for transforms with thousands of parameters, but there it decides whether a registration fits in memory.

TBB still trails Pool on BCD (52% on main), mid-sagittal plane alignment (28%), and the ANTS normalization tests (1.5 to 2 times). Under TBB the affine metric used 8 to 15 times the CPU of Pool at about the same wall time (`data/mattes_wu_sweep_sparse0002_20261102.txt`), which points to oneTBB workers spinning between short, back-to-back parallel regions. These gaps are outside the scope of the three changes.

The changes carry three known risks. `ParallelizeArray` now hands several indices to one task; the one ITK caller that synchronizes between indices, `ParallelSparseFieldLevelSetImageFilter`, already forces `PlatformMultiThreader` (ITK pull request 6337), so the change does not reach it. Keeping rows whole limits parallelism to the number of rows, as Pool already does. Fewer metric work units change the order of floating-point sums, so metric values can differ in the last bits.

7. *Scaling to large images*

The results above use images of 128^3 voxels or fewer, typical of brain MRI. CT volumes of 512^3 and 1024^3 voxels are common, so the threader microbenchmarks were repeated at 256^3, 512^3, and 1024^3, and Mattes MI v4 at 256^3 and 512^3, on upstream main and on this branch, under TBB and Pool at 48 threads. Each case ran once, after the host was 90% idle (`data/large_images_20261103_*`).

TBB chunking scales with the image. The chunk size is the number of pixels divided by the number of work units, so larger images get larger chunks. With 768 work units the branch made 512 chunks at every size, with the smallest at 2,097,152 pixels for 1024^3; upstream made 2,048 to 7,194 chunks, down to single voxels. Table 6 gives the times. `ParallelizeArray` under upstream TBB falls far behind as the count grows (33 times slower than the branch at 1024^3); the branch tracks Pool. `ParallelizeImageRegion` is limited by memory bandwidth at 512^3 and above, and all variants agree to within about 3%. Resampling ranges from 9% slower (512^3) to 8% faster (1024^3) than upstream, within the noise of a single run.

Kernel	256^3	512^3	1024^3
ParallelizeArray	28,050 / 1,007 (918)	219,093 / 7,659 (7,144)	1,842,551 / 56,396 (52,982)
ParallelizeImageRegion	1,015 / 453 (488)	4,903 / 4,547 (4,500)	37,276 / 38,078 (36,872)
ResampleEuler3D	16,836 / 15,936 (15,546)	118,479 / 128,635 (118,834)	957,376 / 882,109 (919,110)

Table 6: Microseconds per call at 48 threads, one run, upstream TBB / branch TBB (Pool on the branch). Source: `data/summary_large_microbench.csv`.

Setting the work units equal to the threads made large resamples 15% to 35% slower than the default of 768, because 32 to 64 chunks balance poorly across 48 threads; the default work-unit count should stay for image regions.

The sampled-metric cap pays off more on large images (Table 7). A 1% sampled affine metric got 1.5 to 2.1 times faster under TBB. A 1% sampled 3,993-parameter BSpline at 512^3 took 32.9 s and peaked at 182.7 GiB under upstream TBB (36.2 GiB under upstream Pool); on the branch it took 24.2 s and 6.1 GiB under either threader.

Dense metrics keep their slowdown, not a larger one. The dense threader keeps one work unit per slice up to the threader's work-unit count, so with the 32 MiB limit per work unit its buffer memory is bounded by 512 times 32 MiB at 512^3 under TBB.

A dense 3,993-parameter BSpline at 512^3 used 19.4 GiB on the branch against 127.7 GiB upstream under TBB, and 7.9 GiB against 36.1 GiB under Pool, and ran 11.6% and 11.4% slower. That is the same cost measured at 96^3 (7% to 11%), so it does not grow with image size. The metric value was identical in every case.

Size	Case	Threader	Time	Peak RSS, GiB
256^3	Affine, 1%	TBB	13.3 / 6.3 ms	0.6 / 0.6
256^3	BSpline, 1%	TBB	3.22 / 3.02 s	23.6 / 2.5
256^3	BSpline, 1%	Pool	3.12 / 3.01 s	32.9 / 2.6
512^3	Affine, dense	TBB	3.69 / 3.70 s	5.0 / 5.0
512^3	Affine, 1%	TBB	70.0 / 45.9 ms	5.0 / 5.0
512^3	BSpline, 1%	TBB	32.9 / 24.2 s	182.7 / 6.1
512^3	BSpline, 1%	Pool	32.7 / 24.9 s	36.2 / 6.1
512^3	BSpline, dense	TBB	2,106 / 2,350 s	127.7 / 19.4
512^3	BSpline, dense	Pool	2,094 / 2,332 s	36.1 / 7.9

Table 7: Mattes MI v4 per `GetValueAndDerivative` at 48 threads, one run, upstream / branch. Dense 512^3 BSpline rows time one evaluation without a warm-up. Source: `data/summary_large_mattes.csv`.

The thresholds do not need to depend on problem size. The chunk size already scales with the image, and the sampled cap does not depend on it. The only size-dependent quantity is dense Mattes buffer memory, which grows with the slice count up to the work-unit count: about 16 GiB of buffers at 512^3 under TBB as measured, and at most 24 GiB at 1024^3 (computed, not measured; upstream would need about 500 GB). A total-budget rule, such as the larger of 4 MiB and a total budget divided by the work units, would bound that further, but only by slowing dense evaluations more: 4 MiB per work unit cost about 20% at 96^3. Given a dense cost that stays near 11% at every size and memory 4.6 to 6.6 times below upstream, a fixed 32 MiB per work unit is the better trade, and the dense bound is documented here.

8. Automatic derivative buffer size

The three changes above merged into ITK as pull request 7097 (`3c3e84bc9ab`). Their fixed 32 MiB limit makes dense BSpline evaluations about 11% slower. A follow-up change (`src/itk-patches-autobuffer/`, commit `c5602f79836` on `3c3e84bc9ab`) adds `SetMaximumDerivativeBufferBytesPerWorkUnit()`. Its default, zero, sizes the buffer as the smaller of the bytes the work unit's samples can fill and a quarter of physical memory divided by the work units, kept between 32 MiB and 1 GiB; a nonzero value is used as given. The rule is also the public static function `ComputeDefaultDerivativeBufferBytesPerWorkUnit(samples, parameters, workUnits, physicalMemoryBytes, memoryFraction)`, so a caller can size the buffers for a different machine; for example, a 64 GiB host gives a dense 512^3, 512-work-unit BSpline the 32 MiB floor, and a 256 GiB host gives it 128 MiB. On this 251 GB host the rule gives dense 96^3 work units 8,000 rows, the upstream maximum, and dense 512^3 TBB work units 4,000 rows.

Time and memory. Table 8 compares upstream, the fixed 32 MiB limit, and the automatic size at 48 threads (96^3: median of three; 512^3: one cold evaluation). The automatic size removes the dense slowdown: dense evaluations come within 3.4% of upstream, against 10% to 13% slower with 32 MiB in this run. On this host it does so by returning dense memory to upstream levels at 96^3 (24.6 and 58.9 GiB against 3.0 and 3.3 GiB) and to half of upstream at 512^3 under TBB (66.6 against 127.7 GiB). Sampled evaluations keep the memory reduction (2.0 to 2.2 GiB against 9.2 to 29.4 GiB upstream).

Case	Threader	Time, s	Peak RSS, GiB
96^3 BSpline 3,993, dense	TBB	13.9 / 15.7 / 14.4	24.5 / 3.0 / 24.6
96^3 BSpline 3,993, dense	Pool	14.4 / 15.9 / 14.3	24.7 / 3.0 / 24.7
96^3 BSpline 10,125, dense	TBB	35.7 / 40.1 / 35.7	58.7 / 3.3 / 58.9
96^3 BSpline 10,125, dense	Pool	35.8 / 40.4 / 34.6	58.8 / 3.3 / 59.1
96^3 BSpline 3,993, 5%	TBB	1.01 / 0.87 / 0.88	11.6 / 2.1 / 2.1
96^3 BSpline 3,993, 5%	Pool	0.89 / 0.88 / 1.01	9.2 / 2.1 / 2.2
96^3 BSpline 10,125, 5%	TBB	2.24 / 2.19 / 2.11	29.4 / 1.7 / 2.0
512^3 BSpline 3,993, dense	TBB	2,106 / 2,350 / 2,062	127.7 / 19.4 / 66.6
512^3 BSpline 3,993, dense	Pool	2,094 / 2,332 / 2,105	36.1 / 7.9 / 36.1

Table 8: Mattes MI v4 per `GetValueAndDerivative`, upstream / fixed 32 MiB / automatic, 48 threads. Source: `data/summary_autobuffer.csv` and `data/summary_large_mattes.csv`.

Variance. With the automatic size the sampled 3,993-parameter case is bimodal: each run took about 0.87 s or about 1.02 s (two of three Pool runs and one of three TBB runs were slow), while the fixed 32 MiB limit stayed between 0.86 and 0.89 s. Larger buffers flush less often but hold the shared lock longer per flush (up to about 250 MB of rows), so other work units wait or grow their own buffers.

Tests. On `3c3e84bc9ab`, 811 of 811 tests in ITKCommon, ITKMetricsv4, ITKOptimizersv4, ITKRegistrationMethodsv4, and ITKLevelSetsv4 pass under TBB and Pool, including eight GTests for the setting: the default, an explicit value, the bounds, the floor, a derivative that does not depend on the buffer size, and the static rule for a given memory size, fraction, and the host. In the forest, ANTs (185 of 185), BRAINSTools (452 of 455), and elastix (162 of 164) gave the same passing and failing tests as the base commit under both threaders (`data/forest_auto/`); that run used the change before it was rebased onto the merge and before the rule was made static, which changed neither the computed sizes nor the code path. The timings above were measured with that same version.

Assessment. The automatic size trades memory for time according to the host, which recovers the dense speed on a large workstation but removes the dense memory reduction there and adds run-to-run variance for sampled evaluations. Callers that need the memory bound can set 32 MiB, or size for a smaller machine

with the static rule. The slowdown comes from contention on one shared lock, so methods that reduce that contention, such as per-work-unit arrays merged in order (pull requests 6622 and 7089) or a tree reduction (draft 6765), may recover the time without the memory. These remain to be compared.

9. Related work

The same failure modes have been reported independently. On the ITK forum (topic 7736, 2026), itk-elastix ran about 8 times slower than the elastix executable; the cause was traced to TBB's default of 1,024 work units on 64 threads combined with per-work-unit Mattes histograms. ITK issue 3591 reports a TBB build about 10 times slower on an AMD host, and issue 6362 notes that `TBBMultiThreader::SetNumberOfWorkUnits` ignores the global thread maximum. Issue 950 proposed a per-filter ratio of work units to threads in 2019.

Two open ITK pull requests touch the same Mattes code. Pull request 7089 (carrying 6622) makes Mattes derivatives reproducible by giving each work unit a full joint-PDF derivative array and removes the buffer that the second change here bounds; if it lands first, that change is superseded. Its memory grows with the number of work units, so the first change here bounds it for sampled metrics. Pull request 7084 changes the local-support derivative pass, and draft 6765 prototypes a general reduction framework.

10. Reproducing the results

- `src/itk-patches/` holds the three ITK commits, to apply on ITK main `fd19f432272`.
- `src/` holds the two benchmark programs and their `CMakeLists.txt`.
- `scripts/` holds the harnesses that produced every file in `data/`, and `scripts/make_figures.py` rebuilds every figure and summary table from `data/` without running a benchmark.
- `data/` holds the raw measurements, including run start times and idle fractions, and the exclusion windows.

Timings depend on the host; rerun the harnesses on an idle machine and compare ratios between configurations rather than absolute seconds.